

Introduction aux Structures de Données et Collections en Java

Ing. ASSOU Yao Mokpokpo

2025-03-05

Illustration



Une collection est un objet qui regroupe plusieurs éléments dans une structure unique, facilitant leur manipulation (ajout, suppression, recherche, etc.).

Importance d'utiliser les collections

Les collections sont vraiment important pour développer les logiciels en entreprise.

① Gestion efficace de données

- Les applications récupèrent les données (**immenses**) des bases de données.
- Ces données massives doivent être traitées dans l'application et permettent au utilisateur de faire un bon usage
- Disponible des différents structures pour satisfaire le client
- Optimiser la mémoire et la performance des applications

Importance d'utiliser les collections

② Abstraction et réutilisabilité

- Réduisent la complexité du code en utilisant des méthodes prédéfinies

Importance d'utiliser les collections

③ Performance

- Algorithmes de tri et de recherche intégrés
- Adapatabilité aux besoins de montée en charge des applications d'entreprise

Importance d'utiliser les collections

- ④ Les collections couramment utilisés
 - ArrayList : Listes dynamiques
 - HashMap : Stockage clé-valeur
 - LinkedList : Manipulations fréquentes
 - TreeSet : Données triées
 - Queue : Files d'attente de traitements

Définition

Une collection ordonnée qui permet l'enregistrement des éléments dupliqués et l'accès par index. Caractéristiques principales :

Caractéristique

- 1 Maintient l'ordre d'insertion
- 2 Autorise les doublons
- 3 Permet l'accès par position (index)
- 4 Implémentations courantes : ArrayList, LinkedList
- 5 Indexation commence à 0
- 6 Permet l'insertion et la suppression d'éléments à des positions spécifiques

Définition

Une collection qui ne contient pas de doublons et ne préserve pas nécessairement l'ordre. Caractéristiques principales :

Caractéristiques

- 1 Rejette les éléments dupliqués
- 2 Ne permet pas l'accès par index
- 3 Utile pour garantir l'unicité des éléments
- 4 Implémentations principales : HashSet (non ordonné), TreeSet (trié)
- 5 Basé sur la méthode equals() pour identifier les doublons
- 6 Très rapide pour les opérations de recherche

Définition

Une collection qui stocke des paires clé-valeur, où chaque clé est unique.
Caractéristiques principales :

Caractéristiques

- 1 Associe une clé unique à une valeur
- 2 Chaque clé ne peut apparaître qu'une seule fois
- 3 Permet de récupérer, ajouter, modifier des valeurs par leur clé
- 4 Implémentations courantes : HashMap (non ordonné), TreeMap (trié)
- 5 Méthodes clés : put(), get(), remove(), containsKey()
- 6 Très utilisé pour créer des dictionnaires ou des caches

Comparaison

Caractéristique	List	Set	Map
Définition	Collection ordonnée d'éléments	Collection d'éléments uniques	Collection de paires clé-valeur
Permet les doublons	✓ Oui	✗ Non	✓ Uniquement pour les valeurs (pas les clés)
Implémentations principales	✓ ArrayList , LinkedList	✓ HashSet , LinkedHashSet , TreeSet	✓ HashMap , LinkedHashMap , TreeMap
Utilisation typique	✓ Stocker des éléments ordonnés avec doublons	✓ Stocker des éléments uniques	✓ Associer des clés à des valeurs

Figure 2: Comparaison

Liste

boolean	add(E e) Appends the specified element to the end of this list (optional operation).
void	add(int index, E element) Inserts the specified element at the specified position in this list (optional operation).
boolean	addAll(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator (optional operation).
boolean	addAll(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list at the specified position (optional operation).
void	clear() Removes all of the elements from this list (optional operation).
boolean	contains(Object o) Returns true if this list contains the specified element.
boolean	containsAll(Collection<?> c) Returns true if this list contains all of the elements of the specified collection.
boolean	equals(Object o) Compares the specified object with this list for equality.

Figure 3: Les listes

Liste

E	get(int index) Returns the element at the specified position in this list.
int	hashCode() Returns the hash code value for this list.
int	indexOf(Object o) Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	isEmpty() Returns true if this list contains no elements.
Iterator<E>	iterator() Returns an iterator over the elements in this list in proper sequence.
int	lastIndexOf(Object o) Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
ListIterator<E>	listIterator() Returns a list iterator over the elements in this list (in proper sequence).
ListIterator<E>	listIterator(int index) Returns a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list.
E	remove(int index) Removes the element at the specified position in this list (optional operation).
boolean	remove(Object o) Removes the first occurrence of the specified element from this list, if it is present (optional operation).
boolean	removeAll(Collection<? c> c) Removes from this list all of its elements that are contained in the specified collection (optional operation).

Figure 4: Les listes

Liste

boolean	remove(Object o) Removes the first occurrence of the specified element from this list, if it is present (optional operation).
boolean	removeAll(Collection<?> c) Removes from this list all of its elements that are contained in the specified collection (optional operation).
default void	replaceAll(UnaryOperator<E> operator) Replaces each element of this list with the result of applying the operator to that element.
boolean	retainAll(Collection<?> c) Retains only the elements in this list that are contained in the specified collection (optional operation).
E	set(int index, E element) Replaces the element at the specified position in this list with the specified element (optional operation).
int	size() Returns the number of elements in this list.
default void	sort(Comparator<? super E> c) Sorts this list according to the order induced by the specified Comparator .
default Splititerator<E>	splititerator() Creates a Splititerator over the elements in this list.
List<E>	subList(int fromIndex, int toIndex) Returns a view of the portion of this list between the specified <i>fromIndex</i> , inclusive, and <i>toIndex</i> , exclusive.
Object[]	toArray() Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	toArray(T[] a) Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.

Figure 5: Les listes

Initialisation du code

```
private List<Etudiant> etudiants;

public EtudiantService() {
    this.etudiants = new ArrayList();
}

public EtudiantService(List<Etudiant> etudiants) {
    this.etudiants = etudiants;
}
```

Ajouter un étudiant

```
public Etudiant ajouter(Etudiant etudiant) {  
    boolean resultat = this.etudiants.add(etudiant);  
    if(resultat) {  
        return etudiant;  
    }  
    return null;  
}
```


Supprimer un étudiant

```
public boolean supprimer(Etudiant etudiant){  
    return this.etudiants.remove(etudiant);  
}
```

Avoir la liste des etudiants

```
public ArrayList<Etudiant> etudiants() {  
    return (ArrayList<Etudiant>) this.etudiants;  
}
```

Rechercher un étudiant

```
public Etudiant rechercher(String nom) {  
    List<String> noms = new ArrayList();  
    Etudiant etudiant = null ;  
    // Chargement  
    for(Etudiant etud : etudiants) {  
        noms.add(etud.getNom());  
    }  
    // Get age if nom existe  
    if(noms.contains(nom)) {  
        int index = noms.indexOf(nom);  
        etudiant = (Etudiant) this.etudiants.get(index);  
    }  
    return etudiant;  
}
```